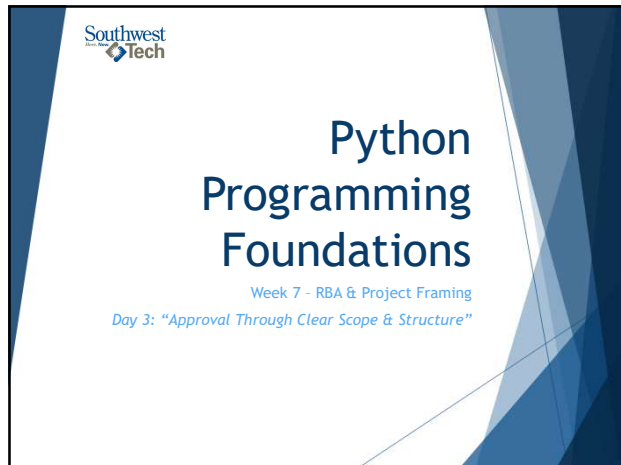




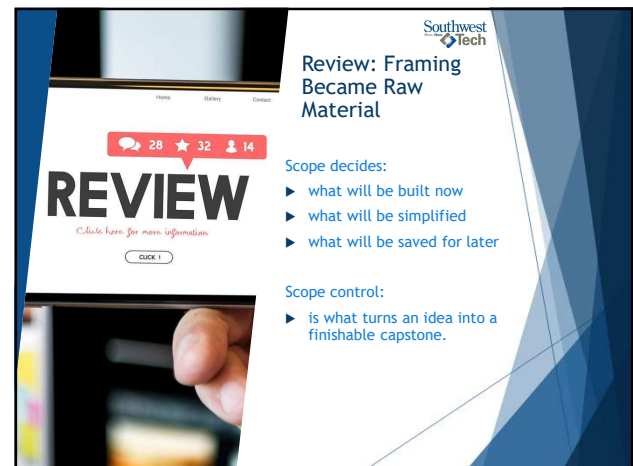
1



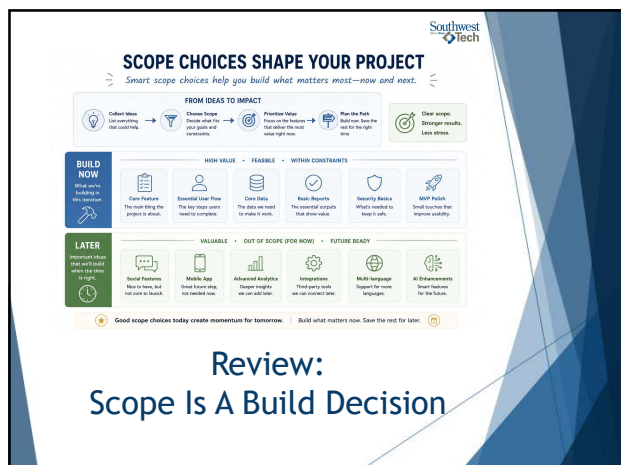
2



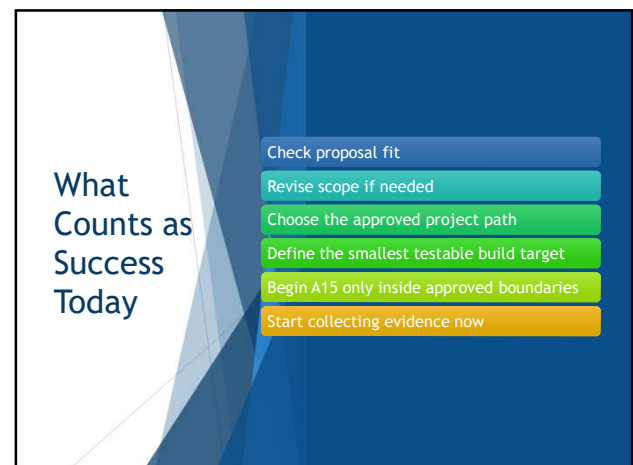
3



4



5



6



Today's Success

7

What We Will Use Today

- ▶ approval criteria
- ▶ project path comparison
- ▶ first build target
- ▶ run instructions
- ▶ validation evidence
- ▶ AI accountability

8



Toolbox

9

What We Will "Skip" For Now

- ▶ final polish
- ▶ presentation rehearsal
- ▶ optional features
- ▶ unapproved expansions
- ▶ professional packaging

Today we approve the scope and start the smallest useful build.

10



Parked For Later

11

Project Approval/Revision

12



Approval Criteria

An approvable proposal shows:

- ▶ clear purpose
- ▶ realistic features
- ▶ expected inputs and outputs
- ▶ likely structure
- ▶ risks or unknowns
- ▶ AI-use plan
- ▶ why the scope is realistic

13



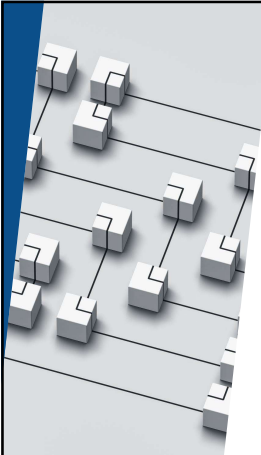
PROPOSAL REVIEW CHECKLIST

Use this checklist to make sure your capstone idea is clear, realistic, and ready for approval.

1. PURPOSE	- What problem or need does this project address? - Who is it for and why does it matter?	☐ Clear and specific ☐ Focused and achievable
2. REALISTIC FEATURES	- What core features will this project include? - Are these features achievable with my current skills and time?	☐ Clear and variable ☐ Organized and logical
3. INPUTS AND OUTPUTS	- What inputs will the project accept? - What outputs or results will it produce?	☐ Identified and planned ☐ Responsible and transparent
4. STRUCTURE	- What is the overall structure or approach? - What technologies or tools will I use?	☐ Right-sized for success ☐ Ready for approval? Review each item and confirm with confidence.
5. RISKS	- What could go wrong or cause delay? - What is my plan if a risk occurs?	☐
6. AI-USE PLAN	- How will I use AI to support my work? - How will I verify, revise, and take responsibility?	☐
7. REALISTIC SCORE	- Can I complete this within the time available? - Have I removed or parked anything out of scope?	☐

A strong proposal leads to a strong project. Clear, focused, and realistic. Successful at the end.

14



Project Path Changes Complexity

Possible capstone paths:

- ▶ console utility
- ▶ data-driven tool
- ▶ file-based tracker
- ▶ API-style fetcher
- ▶ light architecture-preview project, if approved

Each path carries different risks.

15



PROJECT PATH COMPARISON

Five approved paths. Choose the one that fits your idea, skills, and time.

	1. CONSOLE UTILITY	2. DATA-DRIVEN TOOL	3. FILE TRACKER	4. API FETCHER	5. ARCHITECTURE PREVIEW
What it is	A program that runs in the console and solves a specific task.	A tool that works with data in files, APIs, or simple structures.	A tool that reads, writes, and organizes data in a folder.	A tool that gets data from an external API and does useful results.	A small project that shows the flow of a larger system.
Example idea	Budget calculator with menu.	Course analyzer from a data file.	Notes tracker with search.	Weather lookup tool.	Preview of a full application flow.
Core focus	Logic, input handling, program flow.	Data handling, calculations, search, summaries.	File I/O, persistence, organization.	API calls, data parsing, error handling.	Structure, components, connections.
Typical skills used	Variables, conditions, loops, functions, namespaces.	Lists, dictionaries, loops, functions, summarizing data.	File read/write, data validation, simple menus.	Requests, JSON, data mapping, basic error handling.	Modular design, forms, interfaces, data flow diagrams.
Final build target	A working menu with one-path feature.	Load data and show one meaningful report.	Add one item and save it successfully.	Fetch data and display it clearly.	Show the components and how they connect.
What to watch	- Feature creep - Too many features - Weak input validation	- Overcomplicated data - Hard to explain logic - Missing edge cases	- File format issues - Data loss on handling - Flaky error handling	- API limits or changes - Network errors - Unstable data mapping	- Too much design - Not building anything - Hard to explain value

Any path can lead to a strong capstone. Pick the one that fits, keep it focused, and build with purpose.

Choose your path. Define your first target. Use the first easy target.

16



Can This Be Finished?

Ask:

- ▶ What is the smallest working version?
- ▶ What can be tested first?
- ▶ What can be removed if time gets tight?
- ▶ What must be true for this to count as working?

17



FINISHABILITY: BUILD WHAT YOU CAN FINISH

A clear boundary today helps you deliver a working project.

SMALL, TESTABLE CORE

Build this now.

- Core logic that solves the main problem
- Basic input or output menu
- Clear output or result
- Works end to end

Goal: Prove the idea works with a simple, complete experience.

LATER FEATURES

Save these for after the core works.

- Advanced options and settings
- Reports and visualizations
- Cloud sync or external integrations
- Pretty themes and extra content

Goal: Add value and polish after the core is stable and testable.

CHECK YOUR PLAN

1. SMALLEST WORKING VERSION?

What is the smallest thing I can build that still proves the idea works?

2. TEST FIRST?

Can I test this core early to confirm I'm on the right path before adding more?

3. REMOVE IF TIGHT?

What features can I remove or delay if time or scope gets tight?

4. COUNTS AS WORKING?

Does the core deliver real value and solve the main problem for the user?

You don't have to build everything. Finish a focused core. Then make it even better.

Finishability is a feature. A finished core is better than a perfect plan.

18



Can This Be Explained?

Ask:

- ▶ Can I describe the main logic?
- ▶ Can I explain the data?
- ▶ Can I show validation evidence?
- ▶ Can I explain AI contributions, if any?

19



EXPLAINABILITY CHECK

Before you submit, make sure you can explain your project clearly.

1. MAIN LOGIC	2. DATA	3. VALIDATION EVIDENCE	4. AI CONTRIBUTION
- What problem does it solve? - How does the logic work? - What happens step by step? Example: Calculates total using a loop and conditions.	- What data does it use? - Where does it come from? - How is it organized? Example: Reads items from a file and stores in a list.	- What did you test? - What is the output or result? - How do you know it works? Example: Tested with 3 cases and got the expected results.	- What did AI help you with? - What did you change? - What did you learn? Example: AI helped with code structure to merge the logic.

MY EXPLANATION
I can explain how my program works, what data it uses, how I tested it, and how I used AI responsibly.

★ If you can explain it in your own words, you're ready. ✓ Clear thinking. Clear project. Confident submission.

Can This Be Explained?

20

Revision Is Not Rejection

If the proposal needs revision, that is not failure.

Targeted revision protects the student from building something too large, unclear, or hard to explain.

21



THE REVISION LOOP

Improve the idea. Strengthen the plan. Increase your confidence.

1. PROPOSAL DRAFT	2. FEEDBACK RECEIVED	3. REFLECT AND REFINE
You start with an idea and a first draft. Idea: Build a productivity tool. Key features: task list, alerts, reminders, sharing. Jargon: Full app with user accounts and cloud sync.	Instructor and peers give questions and suggestions. - Scope may be too large for the term. - Focus on the core problem. - Clarify inputs and outputs. - What is the first working version?	You review the feedback and rethink the plan. - What is the main problem to solve? - What is the product called now? - Which features add real value now? - What can be saved for later?

ITERATE WITH PURPOSE
Each cycle makes the project clearer, more focused, and more achievable.

4. CLEANER, STRONGER PLAN	5. UPDATED SCOPE
The revised plan is focused, realistic, and ready to build. - Problem is clear and specific. - Scope fits time and skills. - First build target is defined. - Path to success is visible.	You adjust the plan based on what matters most. Focus: Improved task manager. Core features: add task, list tasks, mark complete, delete task. Scope: Local app, single user, no cloud.

Revision protects the project.
Good ideas are not perfect on the first try. Strong projects are shaped through revision.

Revision Is Not Rejection

22



Demo #1

"Capstone Proposal Example"

23



What to Watch For

- ▶ clear purpose
- ▶ focused feature list
- ▶ structure choice
- ▶ risk notes
- ▶ AI-use boundary
- ▶ realistic scope

24



Capstone Project Proposal

Project Title: Personal Task Tracker Student: Your Name

1. PURPOSE
Build a simple app that helps users add, track, and complete tasks with basic data.

2. FOCUSED FEATURES
• Add a task
• List tasks
• Mark task complete
• Show simple stats (total, completed)

3. STRUCTURE
Input: Add task form → Single: Show task (on screen) → Data: List of tasks (in memory) → Output: Task list and stats

4. RISK NOTES
• Scope may grow beyond core features.
• Time limit of 6 weeks requires an ability to focus on the most important features.
• Data loss risk if saving is added too early.

5. AN OUTBOUND
• An app that helps with task structure, examples, integrating ideas.
• Task notes for logs, data, and explain my work.

6. REALISTIC SCOPE
• One user with one account.
• Personal or shared account.
• Add/modify/delete tasks in memory.
• Basic interface (console or simple UI).

7. FIRST BUILD TARGET
Add a task, list tasks, mark complete, show stats - all working in the UI.

What to Watch For
The same idea can sometimes be built several ways.
Approval depends on choosing the version that fits the course, the time, and your ability to explain it.

25



Demo #2

"Structure Options & Approval Fit"

26



What to Watch For

The same idea can sometimes be built several ways.
Approval depends on choosing the version that fits the course, the time, and your ability to explain it.

27



STRUCTURE FIT: SAME IDEA, SEVERAL PATHS

One idea can be built in different ways. Approval selects the path that fits your time, your course, and your ability to explain it clearly.

Core Idea (Same in every path):	A. CONSOLE UTILITY	B. DATA-DRIVEN TOOL	C. FILE TRACKER	D. API FETCHER	E. ARCHITECTURE PREVIEW
How it works	User interacts in the console; program processes input and shows results.	Data stored in in-memory; program processes input and shows results.	Data read from/to files; data stored in memory; results shown.	Program requests data from external API and displays results.	Show how components connect; design or screen mockups.
Data handled	In memory (built-in)	In memory (built-in)	In local files (e.g., JSON)	From external service (API request)	Example data or mock data
Complexity	Lower	Moderate	Moderate	Moderate	Moderate
Time to first working version	Fast	Moderate	Moderate	Moderate	Moderate
Explainability (the best)	Easy to explain step by step	Easy with clear data flow	Easy with clear file flow	Easy if API calls are clear and results are shown	Easy when components and connections are clear
What you learn	Logic, input, output, conditions, loops	Data structures, operations, simple reporting	File I/O, persistence, validation	APIs, data formats, error handling	Structure, components, data flow, design thinking

APPROVAL SELECTS THE PATH THAT FITS
YOUR TIME
YOUR COURSE
YOUR EXPLAINABILITY

Different paths. Same goal. The right choice is the one you can complete and explain.

Demo 2: Structure Options & Approval Fit

28



Assignment #14 Checkpoint

For A14, submit or revise the proposal until the approved scope is clear.
Do not begin major build work outside the approved or provisionally approved scope.

29



CHECKPOINT: WHERE IS YOUR PROPOSAL?

This checkpoint helps you know your next step.

1. APPROVED

Your proposal is clear, focused, and ready to move forward.

What this means:

- Purpose is clear
- Features are focused and realistic
- Structure fits the idea
- Risks are understood
- At boundary is defined
- Scope fits time and course

Next Step: Start building the core. Test early and keep it simple.

2. PROVISIONALLY APPROVED

The idea is strong, but a few areas need adjustment.

What this means:

- Most parts are clear
- Some features may be too large
- Structure needs small changes
- Risks or scope need more detail
- At use or data plan needs clarity

Next Step: Make the needed adjustments. Then move to final approval.

3. REVISE BEFORE MAJOR BUILD

Key parts are missing or unclear. More work is needed before building.

What this means:

- Purpose is unclear or too broad
- Features are too many or not focused
- Structure is missing or unclear
- Risks or scope are not realistic
- At boundary is missing or vague

Next Step: Revise your proposal. Then return for review.

This is not about right or wrong. This checkpoint helps you choose the best next step for a successful project.

Assignment 14 Checkpoint

30

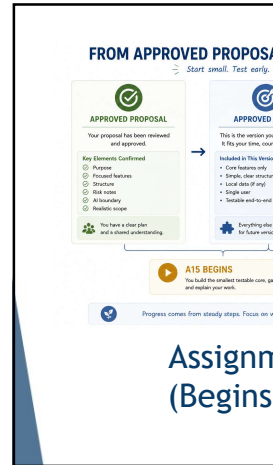


Assignment #15

Once your project is approved or provisionally approved, begin Assignment 15.

Start with the smallest testable core, not the full imagined project

31



FROM APPROVED PROPOSAL TO FIRST BUILD TARGET

Start small. Test early. Grow with confidence.

APPROVED PROPOSAL
Your proposal has been reviewed and approved.

APPROVED SCOPE
This is the version you will build first. It's your time, course, and goals.

SMALLEST TESTABLE CORE
Build the minimum that proves the idea works.

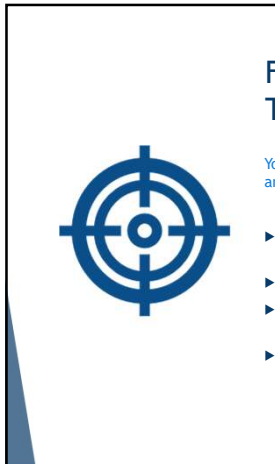
ATIS BEGINS
You build the smallest testable core, gather evidence, and explore your work.

NOT EVERYTHING TODAY
Additional features, improvements, and more are planned for later versions.

Progress comes from steady steps. Focus on what matters now. Build, test, learn, improve.

Assignment 15 (Begins Today)

32

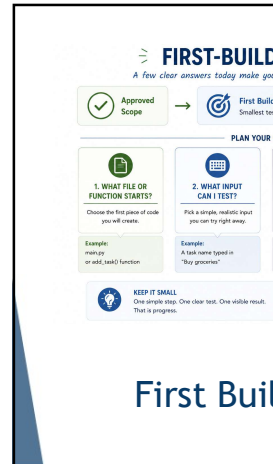


First Build Target

Your first build target should answer:

- ▶ What file or function starts the project?
- ▶ What input can I test?
- ▶ What output proves progress?
- ▶ What will I validate first?

33



FIRST-BUILD PLANNING

A few clear answers today make your first build simple and successful.

APPROVED SCOPE → **FIRST BUILD TARGET** → **ATIS BEGINS**

PLAN YOUR FIRST STEP

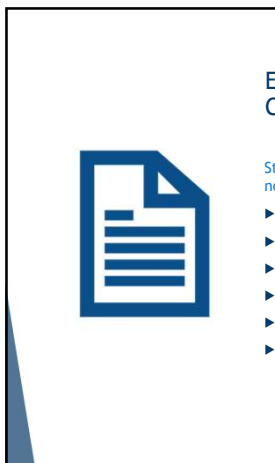
- 1. WHAT FILE OR FUNCTION STARTS?**
Choose the first place of code you can try right away.
Example: main.py or add_test() function
- 2. WHAT INPUT CAN I TEST?**
Pick a simple, realistic input you can try right away.
Example: A test name typed in "Buy groceries"
- 3. WHAT OUTPUT PROVES PROGRESS?**
Decide what you will see that shows it is working.
Example: Text appears in the list
Total tasks = 1
- 4. WHAT VALIDATION FIRST?**
Choose one basic check to keep things reliable.
Example: Text appears in the list
No duplicates allowed

KEEP IT SMALL
One simple step. One clear test. One visible result. That is progress.

YOU CAN GROW IT LATER
Extra features, improvements, and polish come after the core works.

First Build Target

34

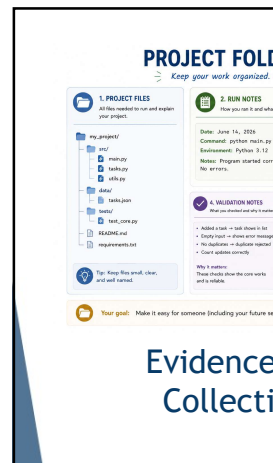


Evidence To Start Collecting Now

Start collecting evidence now:

- ▶ project files
- ▶ run instructions
- ▶ sample input/output
- ▶ validation notes
- ▶ change or revision notes
- ▶ AI-use notes, if used

35



PROJECT FOLDER EVIDENCE

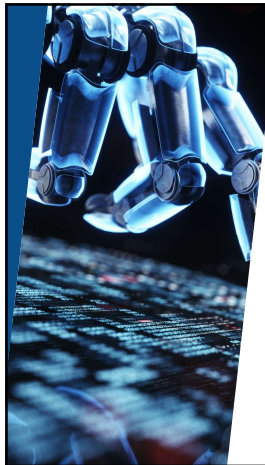
Keep your work organized. Make your thinking visible.

- 1. PROJECT FILES**
All files needed to run and explain your project.
- 2. RUN NOTES**
Date: June 14, 2026
Command: python main.py
Environment: Python 3.12
Notes: Program started correctly. No errors.
- 3. SAMPLE INPUT / OUTPUT**
A small example that shows it works.
- 4. VALIDATION NOTES**
What you checked and why it matters.
- 5. REVISION NOTES**
Changes you made and why.
- 6. AI-USE NOTES**
What AI was used in this project.

Your goal: Make it easy for someone (including your future self) to understand, run, and trust your work.

Evidence To Start Collecting Now

36

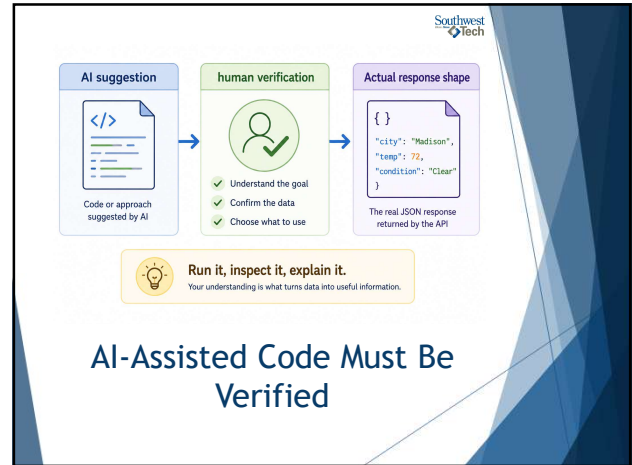


Southwest
Tech

AI Use

- ▶ AI can suggest API code that looks believable but does not match the actual response.
- ▶ You must inspect the response, run the code, and explain what human decision you made.

37



Southwest
Tech

AI suggestion

Code or approach suggested by AI

human verification

- ✓ Understand the goal
- ✓ Confirm the data
- ✓ Choose what to use

Actual response shape

```
{
  "city": "Madison",
  "temp": 72,
  "condition": "Clear"
}
```

The real JSON response returned by the API

Run it, inspect it, explain it.
Your understanding is what turns data into useful information.

AI-Assisted Code Must Be Verified

38



Southwest
Tech

Closing Success Check

By the end of today, you should be able to say:

"My project scope is approved or being revised, and I know the smallest useful piece to build first."

39



Southwest
Tech

CAPSTONE LAUNCH

You have a plan. You have a path. Let's begin.

- 1. APPROVED OR REVISING SCOPE**
Your proposal is reviewed and your scope is clear.
 - ✓ Purpose is clear
 - ✓ Features are focused
 - ✓ Structure fits the step
 - ✓ Risks and boundaries noted
 - ✓ At least boundary is defined
 - ✓ Scope fits time and course
 If revising: Make the changes needed. You control the path forward.
- 2. FIRST USEFUL BUILD TARGET**
You will build the smallest testable core that proves the idea works.
 - Focus on:
 - One simple feature
 - One clear input
 - One visible output
 - One basic validation
 - Goal: Create something useful, measurable, and explainable.
- 3. EVIDENCE COLLECTION STARTED**
Capture the evidence as you build. This shows your thinking and progress.
 - Collect as you go:
 - Project files
 - Build notes
 - Sample text / output
 - Validation notes
 - Revision notes
 - All your notes
 - Why it matters: Evidence explains your work and grows.

YOU'RE LAUNCHING, NOT FINISHING.
This is the beginning of your capstone journey. Build, test, learn, and improve.

PROGRESS OVER PERFECTION.
Small steps today lead to something you're proud of.

Start small. Stay explainable.

40



Southwest
Tech

Thank you!

Have Fun!

41